

README: Group-35-AV-Category-C

Project Overview

This is the README file for the Category C (Deep Learning with Transformers) submission for the COMP34812 Authorship Verification shared task. The solution implements a **two-layer ensemble of four fine-tuned RoBERTa cross-encoder models** with length-and-length-gap-aware bucketed weighting and a long-text model to predict whether two texts were written by the same author.

Group Members: Kanav Gupta, Xiao Ma, and Yangsong Zhou

Model Architecture

The final system is a **two-layer inference pipeline** built on four fine-tuned RoBERTa cross-encoder models.

1. Pre-processing

All four models use **raw-text preservation**: no lowercasing, no URL/email/number normalisation. This preserves surface-level stylistic signals (capitalisation habits, URL formatting, number usage) that are informative for authorship verification. The RoBERTa byte-pair encoding tokenizer is used for model input. Whitespace tokenisation is used separately for computing word counts and length differences for bucket assignment.

2. Cross-Encoder Architecture

All component models share the same **cross-encoder** formulation:

- **Input Encoding:** The two texts (`text_1`, `text_2`) are packed into a single input sequence using the RoBERTa separator tokens: `<s> text_1 </s></s> text_2 </s>`. This joint encoding allows full bidirectional token-level attention between both texts, enabling the model to capture fine-grained stylistic interactions across the pair.
- **Backbone:** `roberta-base` (125M parameters) loaded via Hugging Face `AutoModelForSequenceClassification` with `num_labels=2`.
- **Output:** A 2-class softmax over `[different_author, same_author]`. The probability of class 1 (same author) is used as the model's confidence score.

- **Symmetric Inference:** At test time, both orderings `(text_1, text_2)` and `(text_2, text_1)` are passed through the model and their probabilities are averaged, reducing order-dependent prediction variance.

3. Four Model Variants

The three base models all start from the same anchor checkpoint (Model 1) and diverge through different continuation strategies:

- **Model 1** (`transformer_roberta_20epochs`): Trained from pretrained `roberta-base` for up to 20 epochs with accuracy-based early stopping and swap augmentation (using both pair orderings during training).
- **Model 2** (`transformer_roberta_hard_negative_precision`): Continued from Model 1 for 3 epochs with **hard-negative mining** and **focal loss** ($\gamma=1.5$). Hard-negative mining identifies different-author pairs with high lexical overlap (token Jaccard ≥ 0.35) or matching URL/email/number patterns and upweights their loss by 2.5 $\times$. Focal loss down-weights already-confident predictions, focusing gradient updates on harder, more ambiguous examples.
- **Model 3** (`transformer_roberta_symmetry`): Continued from Model 1 for 3 epochs with **symmetric consistency regularisation** ($\lambda=0.1$). During training, both orderings are passed through the model, and a penalty proportional to $\lambda \times (P_{\text{forward}} - P_{\text{reverse}})^2$ is added to the loss, encouraging order-invariant predictions.
- **Long-text model** (`transformer_roberta_long_text`): Continued from Model 2 for 3 epochs, trained exclusively on long and extra-long text pairs with an extended `max_length=384` (vs. 256 for the base models), addressing the truncation problem on longer documents.

4. Length-and-Length-Gap-Aware Bucketed Ensemble

Rather than using a single global set of ensemble weights, the system assigns each input pair to one of **12 buckets** based on two dimensions:

- **Primary:** Total word count $\rightarrow$ `short` (≤ 120), `medium` (121–200), `long` (201–300), `xlong` (> 300)
- **Secondary:** Absolute per-text length difference $\rightarrow$ `balanced` (≤ 20), `diff` (21–60), `verydiff` (> 60)

Each of the 12 resulting cells (e.g. `short_balanced`, `long_verydiff`) has its own independently optimised weight vector over the 3 base models and classification threshold, found via simplex grid

search on the development set.

5. Long-Text Model Blending

For any pair falling into a `long_*` or `xlong_*` bucket, the final probability is computed as:

$$P_{\text{final}} = (1 - \alpha) \times P_{\text{base_ensemble}} + \alpha \times P_{\text{longtext}}$$

Both α and the threshold are optimised independently per bucket on the development set.

Model Performance

The best system configuration was selected based on accuracy and F1-score on the development set (`dev.csv`). The final results are (to 4 s.f.):

Classification Metrics:

- **Accuracy:** 0.8298
- **Macro Precision:** 0.8399
- **Macro Recall:** 0.8281
- **Macro F1:** 0.8280
- **Weighted Macro Precision:** 0.8388
- **Weighted Macro Recall:** 0.8298
- **Weighted Macro F1:** 0.8283
- **Matthews Correlation Coefficient:** 0.6678

Probabilistic Metrics:

- **ROC AUC:** 0.8579
- **Equal Error Rate (EER):** 0.2251
- **Log Loss:** 0.5624
- **Brier Score:** 0.1904

Confusion Matrix:

	Predicted: Same	Predicted: Different
Actual: Same	2797	259
Actual: Different	761	2176

Code Structure

The code consists of two notebooks:

- `train_Group_35_C.ipynb`: Self-contained Google Colab training notebook containing the full pipeline, including: data loading, preprocessing and feature extraction functions, the cross-encoder training loop (with focal loss, symmetric consistency, hard-negative mining), and checkpoint saving. The notebook trains all four models sequentially in one run by setting `RUN_STAGE = 'selected_final'`.
- `demo_Group_35_C.ipynb`: Inference-only Google Colab notebook. Loads the four pre-trained inference bundles, runs the full ensemble pipeline (per-model inference → bucketed weighted average → long-text model blending → final predictions), and outputs `Group_35_C.csv` in the required submission format. When `dev.csv` is provided as input, it additionally computes and displays evaluation metrics (accuracy, F1, ROC AUC, EER, log loss, Brier score) and a confusion matrix.

Model Resources

The four trained model bundles are required at runtime and are provided as a single zip file on OneDrive: [Download from OneDrive](#)

The four model directories inside the zip are:

- `transformer_roberta_20epochs/` — Model 1 (anchor, 20-epoch raw-text accuracy)
- `transformer_roberta_hard_negative_precision/` — Model 2 (hard-negative + focal loss continuation)
- `transformer_roberta_symmetry/` — Model 3 (symmetric consistency continuation)
- `transformer_roberta_long_text/` — Long-text model (long/xlong pairs, max_length=384)

Each directory contains the fine-tuned model weights and tokenizer files (`hf_model1/`), and an `inference_metadata.json` with the saved config and optimal threshold.

Dependencies

The two notebooks are designed to be run in a standard Google Colab environment. The primary libraries used are:

- `torch` (PyTorch)

- `transformers` (Hugging Face Transformers)
- `accelerate` (required for training only)
- `sentencepiece`
- `scikit-learn`
- `numpy`
- `matplotlib`, `seaborn` (for evaluation plots in the demo notebook)

The install cells at the top of each notebook contain the commands needed to install all dependencies directly.

How to Run

Inference

1. Download `models.zip` from the OneDrive link above.
2. Open `demo_Group_35_C.ipynb` in Google Colab. A CUDA-compatible GPU runtime is recommended.
3. Run the **Installs** cell to install required packages (`transformers`, `sentencepiece`).
4. Upload your test CSV file (e.g. `test.csv`) into `/content/data`.
5. Upload `models.zip` to `/content/` in Colab. The notebook will detect it and extract the four model bundles automatically.
6. In the **Constants** cell, set `INPUT_CSV_NAME` to match your test file name (e.g. `'test.csv'`).
7. Run all remaining cells in order.
8. The final predictions will be saved to `Group_35_C.csv` and downloaded automatically. If the input file contains labels (e.g. `dev.csv`), additional evaluation metrics and a confusion matrix will also be displayed.

Training

1. Open `train_Group_35_C.ipynb` in Google Colab. A CUDA-compatible GPU runtime is strongly recommended (T4 or better). Training on CPU will be extremely slow.
2. Run the **Installs** cell to install required packages (`transformers`, `accelerate`, `sentencepiece`).
3. Upload `train.csv` and `dev.csv` into `/content/data`.
4. In the **Paths and Run Settings** cell, set `RUN_STAGE` to control which models to train. Set it to `'selected_final'` to train all four models sequentially (recommended for full reproduction). Other options include `'raw20ep'` (Model 1 only), `'symmetry'` (Model 3 only), `'hardneg'` (Model 2 only), and `'long_specialist'` (long-text model only).

5. Run all remaining cells in order. The notebook will train the models in the following order:

- Model 1 (`transformer_roberta_20epochs`) — up to 20 epochs from `roberta-base`
- Model 3 (`transformer_roberta_symmetry`) — 3 epochs continued from Model 1
- Model 2 (`transformer_roberta_hard_negative_precision`) — 3 epochs continued from Model 1
- Long-text model (`transformer_roberta_long_text`) — 3 epochs continued from Model 2, on long/long pairs only

Data Sources

The model was trained exclusively on the COMP34812 Authorship Verification `train.csv` dataset and validated and evaluated using the `dev.csv` dataset provided as part of the shared task. No external datasets were used.

The `roberta-base` pretrained weights are downloaded automatically from Hugging Face during training.

Use of Generative AI Tools

Generative AI tools (including Gemini, ChatGPT, and Claude) were used during the development of this coursework in the following transparent ways:

- **Coding Assistant:** Used to help debug PyTorch training loops, optimise data loading pipelines, generate boilerplate code (such as checkpoint saving/loading, CSV I/O, and metric computation), and format evaluation reporting.
- **Writing and Formatting:** Used to structure the Markdown, refine initial drafts, and polish the grammar and phrasing for both the Model Card and this README file to ensure clear communication.